# Chapter 04 공개키 암호(1)

Venture 1st
호서대학교

# 목차

4.1 개요
4.2 배낭암호
4.3 RSA
4.4 디피-헬먼
4.5 타원곡선 암호
4.6 공개키 표기법
4.7 공개키 암호 용도
4.8 인증서와 PKI
4.9 양자 컴퓨터와 공개키
4.10 요약

# 4.1 개요

## ➢ 공개키 암호 방식(Public-key Cryptosystem)

- 비대칭 암호, 이중 키 암호, 또는 비(非)보안키 암호라고도 알려짐
- 공개키 **암호는 암호화와 복호화에 서로 다른 두 개의 키를 사용**
  - 암호화에 사용하는 키를 공개할 수 있으므로, 대칭키 암호 방식의 주요 문제인 키를 안전하게 배분하는 문제를 해결할 수 있음
- 공개키 암호 방식은 수학적 구조를 기반으로 하므로, 새로운 암호 체계를 만드는 것이 어려움
  - 대칭키 암호 방식에 비해 공개키 암호 알고리즘의 수는 아주 적으며, 그 중에서도 극소수만 널리 사용
- 공개키 암호 체계는 '트랩도어 일방향(one-way) 함수'를 기본으로 함
  - **트랩도어 일방향(one-way) 함수** : 함수의 역을 구하는 것이 어려운 일방향 함수이지만 특수한 정보가 있으면 쉽게 역을 구할 수 있는 함수

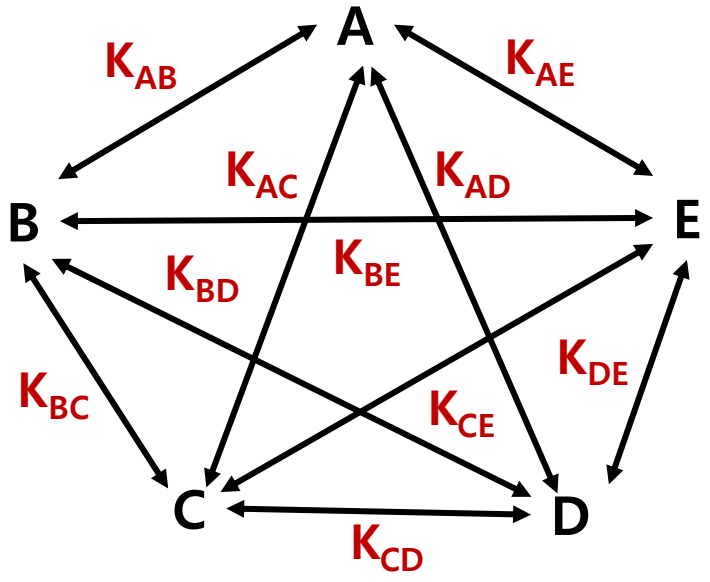
## ➢ 디지털 서명(digital signature)

- 손으로 쓴 자필 서명과 같은 기능을 하는 디지털 정보
- 공개키/개인키를 이용하여 구현 가능
  - 예) 앨리스는 자신의 개인키에 접근할 수 있는 유일한 사람인 동시에 앨리스로 디지털 서명을 할 수 있는 유일한 사람

# 대칭키 암호 기술의 문제점

## (1) 키 분배 및 관리 문제

- 키 분배 문제 : Alice와 Bob 사이에 안전한 채널을 통해 키를 구축해야 함
- 사용되는 키의 수 : n명의 사용자가 있는 네트워크에서 필요한 키의 개수는 **n(n-1)/2**



# 대칭키 암호 기술의 문제점

## (2) 완전한 부인방지 기능 제공 불가

- 암호 통신에 참여한 Alice 또는 Bob이 속이는 행위 방지불가
- Alice가 Bob에게 암호문을 보낸 후에 보내지 않았다고 주장할 수 있음
- Bob과 Alice는 동일한 키를 알고 있으므로 Bob도 동일한 암호문을 생성할 수 있음

(예) Alice가 대칭키 암호 방식으로 암호화 한 주문서를 Bob에게 전송한 후, 나중에 주문하지 않았다고 주장할 수 있음

→ Bob과 Alice는 동일한 키를 알고 있으므로 Bob도 동일한 주문서를 생성할 수 있음

→ Alice가 부인하는 것은 Bob 알지만 제 3자에게 증명할 수 없음

# 공개키 암호 기술의 원리

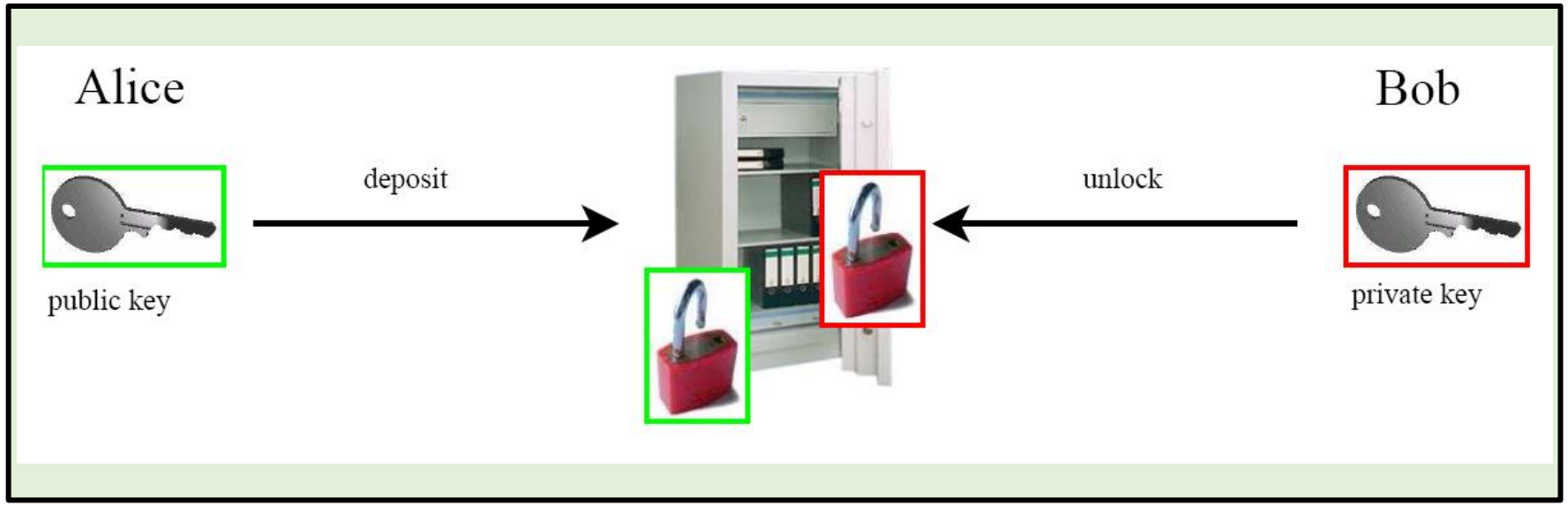
- 1976년에 W. Diffie와 M. Hellman이 IEEE IT-22에 발표한 논문 "New directions in cryptography"에서 공개키 암호시스템의 개념을 처음으로 제안

- 금고를 잠그는 열쇠(암호화 키)는 비밀일 필요가 없음(공개)
- 금고를 여는 열쇠(복호화 키)만 비밀리에 보관하면 됨

- 우편함 원리 이용
  - 모든 사람이 편지를 넣을 수 있음
  - 소유자만이 우편함을 열수 있는 키를 가짐

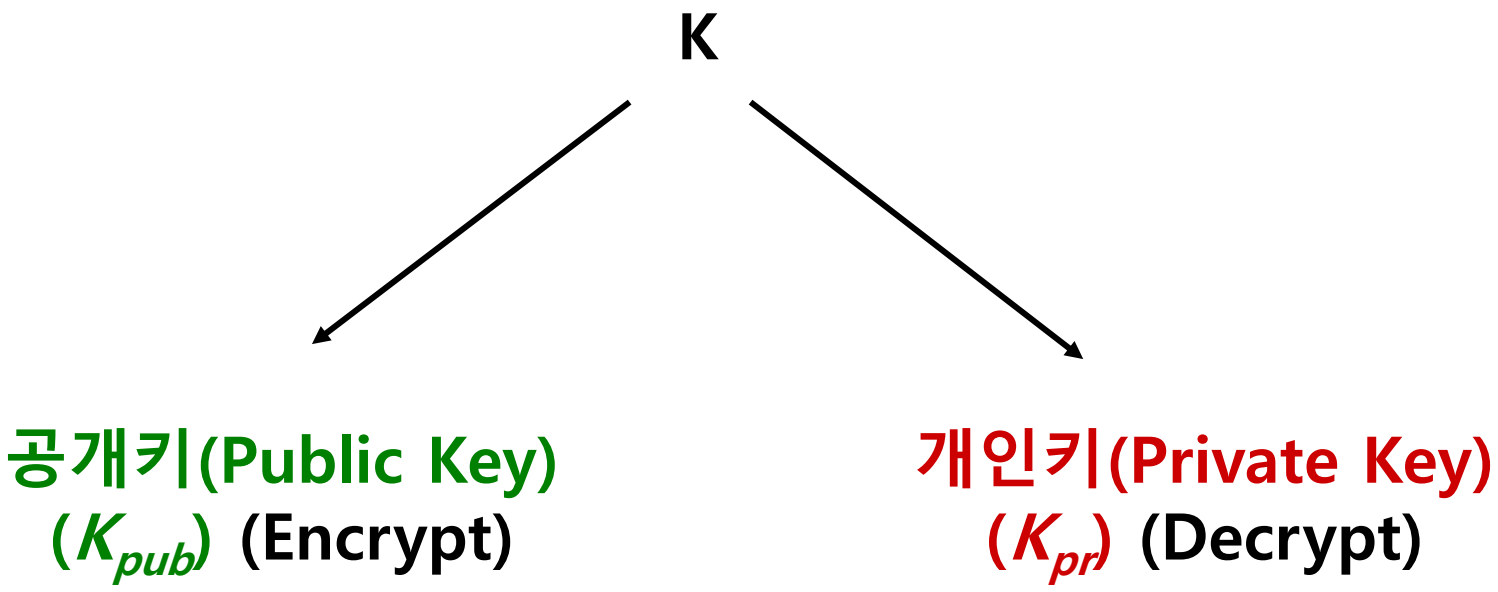
# 공개키 암호 기술의 원리

- 금고를 잠그는 열쇠와 여는 열쇠가 서로 다름
- Alice는 공개열쇠를 이용하여 금고를 잠그고, Bob은 비밀열쇠를 이용하여 금고를 열 수 있음

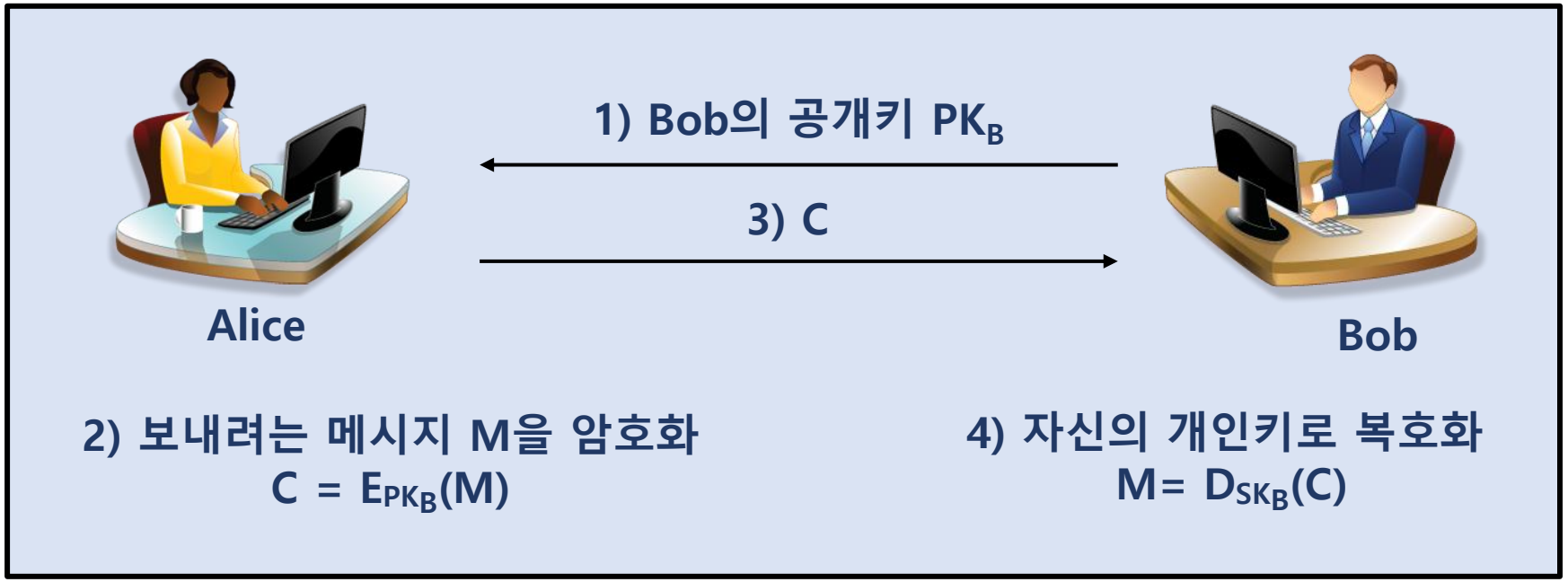


# 공개키 암호 기술의 원리

- 암호화/복호화에 사용하는 키를 다르게 함
  - 키 생성 과정에서 공개키/개인키 키 쌍을 생성함

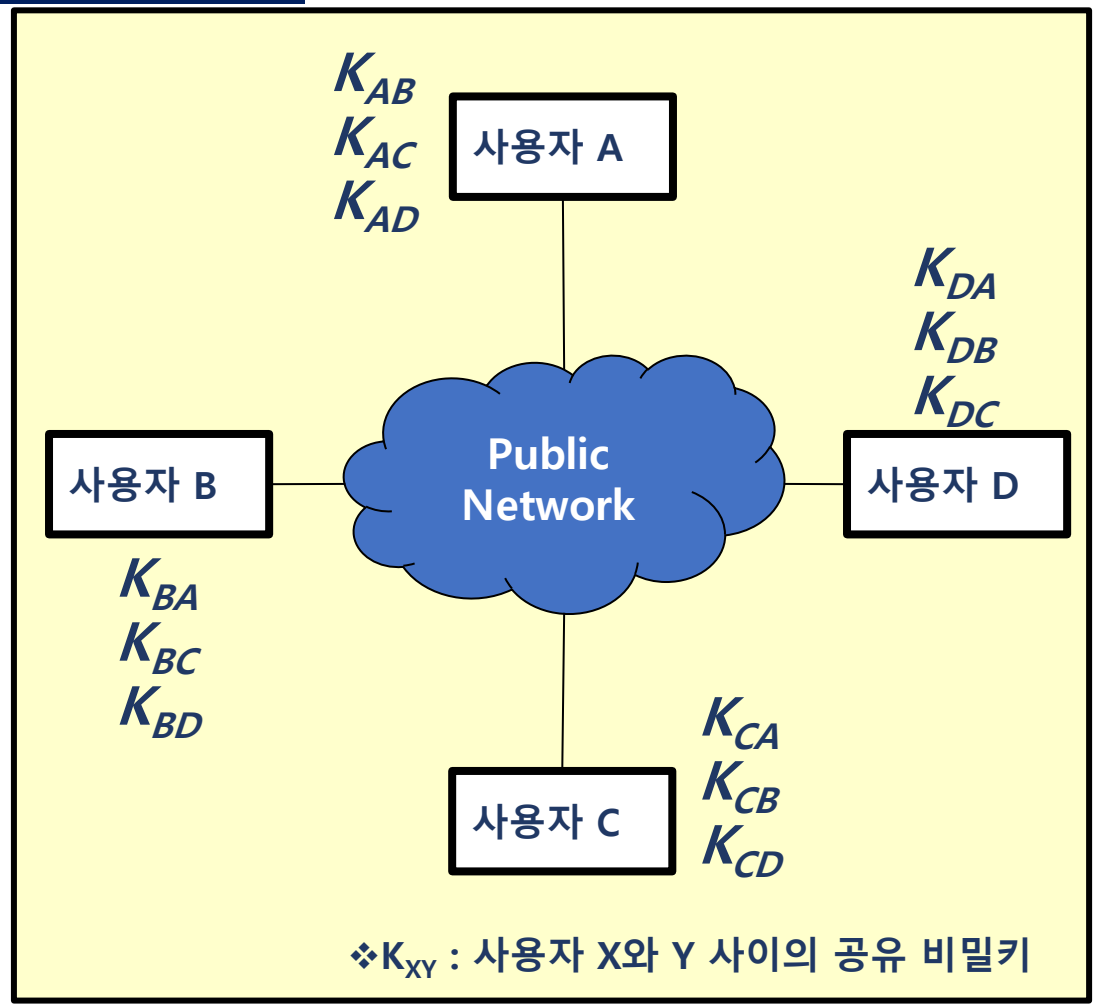


# 공개키 암호 기술의 원리

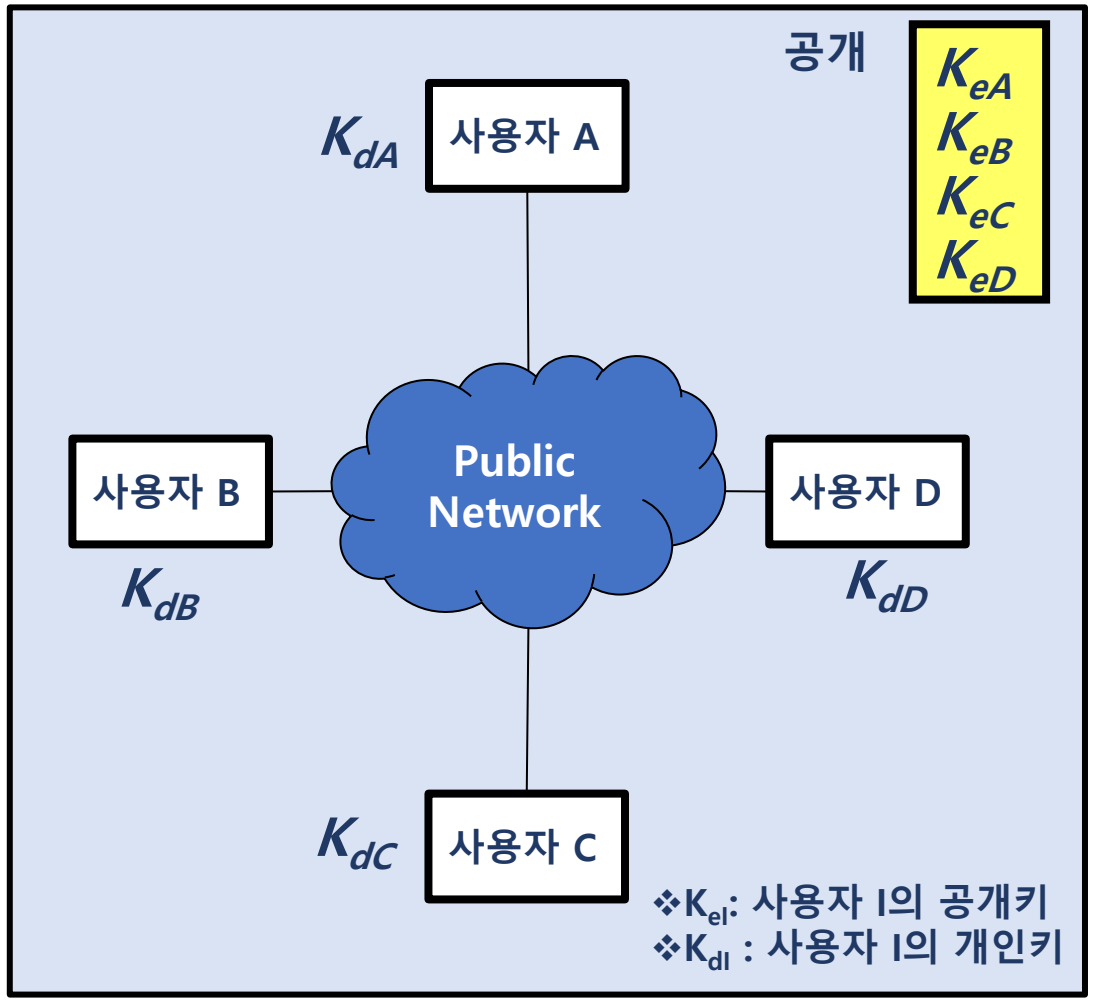



➢ Alice와 Bob은 키를 비밀리에 나누어 가질 필요가 없음 -> **키 분배 문제 해결**

# 암호 시스템의 키 구조




[대칭키 암호 방식]




[공개키 암호 방식]

# 공개키 암호 기술의 활용

## ➢ 공개키 암호 기술이 제공하는 기능

- 암호화(encryption) : 메시지를 암호화하여 기밀성을 제공한다.
- 키 구축(key establishment) : 공개키 암호 기술을 이용하면 Alice와 Bob 사이에 안전하지 않은 채널을 이용하여 안전하게 비밀 공유키를 구축할 수 있다.
- 부인불가(nonrepudiation) : Alice나 Bob이 자신이 생성한 메시지를 나중에 생성하지 않았다고 부인할 수 없도록 하는 기능을 제공한다.(전자서명)
- 식별(identification) : 통신에 참여하고 있는 개체의 신분을 확인할 수 있다.

# 주요 공개키 암호 방식

## ➢ 대표적인 공개키 암호체계

- 배낭암호체계(knapsack cryptosystem) - 1983년 해독
- RSA 암호 방식(소인수분해 문제 이용)
- ElGamal 암호 방식(이산대수 문제 이용)
- 디피-헬먼(Diffie-Hellman) 키 교환 알고리즘(이산대수 문제 이용)
- 타원곡선 암호체계(ECC, Elliptic curve cryptography)(타원곡선 상의 이산대수 문제 이용)

## ➢ 공개키 암호 알고리즘의 보안 수준

| 알고리즘 계열 | 암호 시스템 | 보안수준(비트) | | | |
|---|---|---|---|---|---|
| | | 80 | 128 | 192 | 256 |
| 인수분해 | RSA | 1024 | 3072 | 7680 | 15360 |
| 이산대수 | DH, DSA, ElGamal | 1024 | 3072 | 7680 | 15360 |
| 타원곡선 | ECDH | 160 | 256 | 384 | 512 |
| 대칭키 | AES, 3DES | 80 | 128 | 192 | 256 |

# 4.2 배낭암호

## ➢ 배낭암호 체계(knapsack cryptosystem)

- 머클-헬먼(Merkle-Hellman)이 제안
- 배낭문제는 다음과 같이 표현할 수 있음
  - n개의 배낭에 담을 수 있는 무게가 다음과 같고 목표로 하는 총합이 $S$

    $$W = (W_0, W_1, \ldots, W_{n-1})$$
  - 다음을 만족하는 $a_i \in \{0, 1\}$인 $(a_0, a_1, \ldots, a_{n-1})$를 찾는 문제

    $$S = a_0 W_0 + a_1 W_1 + \ldots + a_{n-1} W_{n-1}$$

  - 예) 배낭 무게 $W$는 다음과 같다고 가정

    $$W = (85, 13, 9, 7, 47, 27, 99, 86)$$
  - 희망하는 무게의 총합인 $S$=172라면 이 중에서 85, 13, 47, 27을 선택(85+13+47+27=172)

    $$a = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7) = (11001100)$$

# 4.2 배낭암호

## ➢ 슈퍼증가(superincreasing) 배낭문제

- 무게가 가장 작은 것에서 가장 큰 것으로 배열할 때 각 무게는 이전의 모든 무게를 더한 것보다 크다는 점만 제외하면 일반적인 배낭문제와 유사

- 예) 슈퍼증가 배낭문제(총합이 $S$=309라고 가정)

  $K$=(3, 6, 11, 25, 46, 95, 200, 411)  (4.1)

  – $a_i$를 구하기 위해 가장 큰 무게부터 가장 작은 무게를 순차적으로 찾아감

    $S < 411$이기 때문에, $a_7=0$

    $S > 200$이므로, $a_6=1$

    $S=S-200=109$

    $S > 95$이므로, $a_5=1$이고 $S=109-95=14$라고 계산

    $a$=10100110

    3+11+95+200=309이므로 해답이 맞는지 쉽게 검증할 수 있음

# 4.2 배낭암호

## ➢ 배낭암호 체계를 구성하기 위해 사용하는 절차

❶ 슈퍼증가 배낭을 생성한다.
❷ 슈퍼증가 배낭을 일반 배낭으로 변환한다.
❸ 공개키는 일반 배낭이 된다
❹ 개인키는 변환 값이 있는 슈퍼증가 배낭키가 된다.

# 4.2 배낭암호

## ➢ 배낭암호 키 생성 과정 예

❶ 다음과 같은 슈퍼증가 배낭을 선택한다.

$$K=(2, 3, 7, 14, 30, 57, 120, 251)$$

❷ 슈퍼증가 배낭을 일반 배낭으로 변환시키기 위해 승수 $m=41$과 계수 $n=491$을 선정

$$\begin{aligned}
2m &= 2\cdot 41 = 82 \pmod{491} \\
3m &= 3\cdot 41 = 123 \pmod{491} \\
7m &= 7\cdot 41 = 287 \pmod{491} \\
14m &= 14\cdot 41 = 83 \pmod{491} \\
30m &= 30\cdot 41 = 248 \pmod{491} \\
57m &= 57\cdot 41 = 373 \pmod{491} \\
120m &= 120\cdot 41 = 10 \pmod{491} \\
251m &= 251\cdot 41 = 471 \pmod{491}
\end{aligned}$$

그 결과로 나타나는 배낭은 (82, 123, 287, 83, 248, 373, 10, 471)이 됨

# 4.2 배낭암호

❸ 공개키는 생성한 일반 배낭이 됨

공개키: (82, 123, 287, 83, 248, 373, 10, 471)

❹ 개인키는 $m^{-1}$ mod $n$인 전환 계수의 곱셈역을 포함하는 슈퍼증가 배낭

개인키: (2, 3, 7, 14, 30, 57, 120, 251), $41^{-1}$(mod 491) =12

# 4.2 배낭암호

## ➢ 배낭암호 방식의 암호화/복호화

① 밥은 앞에서 설명한 방식대로 자신의 공개키/개인키 쌍 생성

- 공개키: (82, 123, 287, 83, 248, 373, 10, 471)
- 개인키: (2, 3, 7, 14, 30, 57, 120, 251), $41^{-1}$(mod 491) =12

② 앨리스는 밥에게 보낼 이진수 메시지 M=10010110을 암호화하고자 함

- 앨리스는 메시지 M 중에서 값이 1인 자리의 공개키 배낭 요소를 선택하고 총합으로 암호문 계산
- 공개키 (82, 123, 287, 83, 248, 373, 10, 471) 중 82, 83, 373, 10 선택
- 암호문 $C$ = 82+83+373+10 = 548

③ 밥은 개인키를 이용하여 암호문을 복호화

- $m^{-1} \cdot C$ (mod $n$)=12·548(mod 491)=193
- 193에 대한 슈퍼증가 개인키 배낭을 품
- 개인키 (2, 3, 7, 14, 30, 57, 120, 251)로 부터 193 = 120 + 57 + 14 + 2를 찾음
- 밥은 $M$=10010110인 메시지를 찾는 것은 쉬움

# 4.2 배낭암호

## ▪ 복호화 과정

$$
\begin{aligned}
548m^{-1} &= 82m^{-1}+83m^{-1}+373m^{-1}+10m^{-1} \\
&= (2m)m^{-1}+(14m)m^{-1}+(57m)m^{-1}+(120m)m^{-1} \\
&= 2(mm^{-1})+14(mm^{-1})+57(mm^{-1})+120(mm^{-1}) \\
&= 2+14+57+120 \\
&= 193 \pmod{491}
\end{aligned}
$$

$$
\begin{aligned}
2m &= 2\cdot 41 = 82 \pmod{491} \\
3m &= 3\cdot 41 = 123 \pmod{491} \\
7m &= 7\cdot 41 = 287 \pmod{491} \\
14m &= 14\cdot 41 = 83 \pmod{491} \\
30m &= 30\cdot 41 = 248 \pmod{491} \\
57m &= 57\cdot 41 = 373 \pmod{491} \\
120m &= 120\cdot 41 = 10 \pmod{491} \\
251m &= 251\cdot 41 = 471 \pmod{491}
\end{aligned}
$$

- $m^{-1}$을 곱함으로써 공개키 배낭 영역에 있는 암호문을 개인키 슈퍼증가 영역으로 변환시킴
- $m^{-1}$ 과 개인키 배낭을 알고있는 밥은 복호가능

# 4.2 배낭암호

- 공격자 트루디가 암호문 값 $C$로 총합이 되는 공개키 요소의 부분 집합을 찾을 수 있다면 개인키 없이 메시지를 풀 수 있음
- 공격자 트루디는 합이 548이 되는 공개키 배낭의 부분 집합을 찾아야만 함
  - 공개키 배낭 $K$=(82, 123, 287, 83, 248, 373, 10, 471)

❖ 배낭 암호 방식은 1983년 애플 II 컴퓨터를 이용하여 Shamir에 의해 해독됨

# 4.3 RSA

## ➢ 등장 배경

- 대칭키 암호 시스템은 **키 교환 및 관리**의 문제가 있음
- 이를 해결하기 위해 공개키 암호 시스템이 연구됨

## ➢ RSA 공개키 암호 방식

- Rivest, Shamir, Adleman(1978년)이 개발
- 소인수분해 문제 이용

# 4.3 RSA

- RSA 암호 시스템의 활용
  - 적은 양의 데이터 암호화에 사용(키 암호화)
  - 전자서명에 주로 사용

- RSA는 AES와 같은 대칭키 암호 기술에 비해 속도가 느리므로, 메시지 암호화에는 거의 사용하지 않음

- AES와 같은 대칭키 암호화에 사용되는 키를 안전하게 교환하기 위해 사용

# 수학적 배경

➢ 정수 집합 Z = { ..., -2, -1, 0, 1, 2, ...}는 다음과 같은 기본 성질을 만족한다.

Z1. 집합 Z위에서 덧셈이 정의된다.

$$a, b \in Z, a + b \in Z$$

Z2. 덧셈에 대하여 교환법칙이 성립한다.

$$a, b \in Z, a + b = b + a$$

Z3. 덧셈에 대하여 결합법칙이 성립한다.

$$a, b, c \in Z, (a + b) + c = a + (b + c)$$

Z4. 특정한 정수 0∈Z은 모든 a∈Z 에 대해 다음을 만족시킨다. 이때 정수 0을 덧셈에 대한 항등원(identity)이라 한다.

$$a + 0 = 0 + a = a$$

Z5. 모든 a∈Z에 대해 다음을 만족하는 –a∈Z 가 존재한다. 이때 정수 –a를 덧셈에 관한 a의 역원(inverse)라 한다. 따라서 정수 집합 Z 위에서는 뺄셈이 정의된다.

$$a + (-a) = (-a) + a = 0$$

# 수학적 배경

Z6. 집합 Z 위에서 곱셈이 정의된다.

$$a, b \in Z, a \cdot b \in Z$$

Z7. 곱셈에 대하여 교환법칙이 성립한다.

$$a, b \in Z, a \cdot b = b \cdot a$$

Z8. 곱셈에 대하여 결합법칙이 성립한다.

$$a, b, c \in Z, (a \cdot b) \cdot c = a \cdot (b \cdot c)$$

Z9. 특정한 정수 $1 \in Z$은 모든 $a \in Z$ 에 대해 다음을 만족시킨다. 이때 정수 1을 곱셈에 대한 항등원(identity)이라 한다.

$$a \cdot 1 = 1 \cdot a = a$$

Z10. 집합 Z위에서 덧셈과 곱셈은 분배 법칙이 성립한다.

$$a, b, c \in Z, a \cdot (b + c) = (a + b) \cdot c = a \cdot b + a \cdot c$$

정수 집합 Z위에서 모든 원소는 덧셈에 대한 역원을 가지고 있으나, 곱셈에 대해서는 1과 -1만 역원을 갖는다. 따라서 정수 집합 Z는 나눗셈이 정의되지 않는다.

# 수학적 배경

## ➢ 나눗셈의 정리

두 정수 $a, b \in Z$가 $a \neq 0$일 때, 다음을 만족하는 $q, r \in Z$이 유일하게 존재한다.

$$b = a \cdot q + r,\ 0 \leq r < |a|$$

두 정수 q, r을 각각 b를 a로 나누었을 때의 몫(Quotient)과 나머지(remainder)라 한다.

# 수학적 배경

- 집합 Z 위의 정수 a와 b에 대해, b = c · a를 만족하는 정수 c가 존재할 때, a는 b의 약수(divisor)라 하고, b를 a의 배수(multiple)이라 한다.

  - 이때, a | b라 표시하고, a는 b를 나눈다 또는 b는 a로 나누어 떨어진다고 한다.

  - 세 정수 a, b, d ∈ Z 에 대하여, 다음을 만족하는 경우 d를 a와 b의 공약수(common divisor)라 한다.

$$d \mid a \ , \ d \mid b$$

  - 두 정수 a, b에 대해 가장 큰 공약수 g를 a와 b의 최대 공약수(gcd : greatest common divisor)라 한다. g = (a, b) 또는 g = gcb(a, b)로 표시한다.

# 수학적 배경

## ➢ 최대 공약수(GCD: Greatest Common Divisor)

- d = gcd(a, b)
- a와 b의 공약수 중 가장 큰 수
- e|a and e|b ⇒ e|d (a와 b의 공약수는 d의 약수)
- gcd(a, b) = 1이면, a와 b는 서로 소(coprime, relatively prime)

## ➢ 최소 공배수(LCM: Least Common Multiple)

- L = lcm(a, b)
- a와 b의 공배수 중 가장 작은 수

# 수학적 배경

## ➢ 두 정수 a, b의 최대 공약수 계산

$b = aq + c, \quad 0 \le c < a$ 일 때,

$d = \gcd(b, a)$ 이고 $d' = \gcd(a, c)$라 하자.

$$
\begin{aligned}
d = \gcd(b, a) &\Rightarrow d \mid b \text{ and } d \mid a \\
&\Rightarrow d \mid c \text{ and } d \mid a \\
&\Rightarrow d \mid d' \\
d' = \gcd(a, c) &\Rightarrow d' \mid a \text{ and } d' \mid c \\
&\Rightarrow d' \mid b \text{ and } d' \mid a \\
&\Rightarrow d' \mid d
\end{aligned}
$$

$\therefore\ d' = d \Rightarrow \gcd(b, a) = \gcd(a, c)$

# 수학적 배경

$b = aq_1 + r_1, \quad 0 \le r_1 < a$ $\qquad (b, a) = (a, r_1)$

$a = r_1q_2 + r_2, \quad 0 \le r_2 < r_1$ $\qquad (a, r_1) = (r_1, r_2)$

$r_1 = r_2q_3 + r_3, \quad 0 \le r_3 < r_2$ $\qquad (r_1, r_2) = (r_2, r_3)$

$\vdots \qquad\qquad \vdots$

$r_{n-2} = r_{n-1}q_n + r_n, \quad 0 \le r_n < r_{n-1}$ $\qquad (r_{n-2}, r_{n-1}) = (r_{n-1}, r_n)$

$r_{n-1} = r_nq_{n+1}$ $\qquad (r_{n-1}, r_n) = r_n$

$\therefore \gcd(a, b) = r_n$

나눗셈을 반복하다가 나머지가 0이 되었을 때, 나누는 수가 최대공약수 임

# 예제

➢ 유클리드 알고리즘을 이용하여 120과 35의 최대 공약수를 구하시오.

| | |
|---|---|
| $120 = 35 \cdot 3 + 15$ | $\gcd(120, 35) = \gcd(35, 15)$ |
| $35 = 15 \cdot 2 + 5$ | $\gcd(35, 15) = \gcd(15, 5)$ |
| $15 = 5 \cdot 3$ | $\gcd(15, 5) = 5$ |

$\therefore \gcd(120, 35) = 5$

# 예제

➢ 유클리드 알고리즘을 이용하여 240과 46의 최대 공약수를 구하시오.

| | |
|---|---|
| $240 = 46 \cdot 5 + 10$ | $\gcd(240, 46) = \gcd(46, 10)$ |
| $46 = 10 \cdot 4 + 6$ | $\gcd(46, 10) = \gcd(10, 6)$ |
| $10 = 6 \cdot 1 + 4$ | $\gcd(10, 6) = \gcd(6, 4)$ |
| $6 = 4 \cdot 1 + 2$ | $\gcd(6, 4) = \gcd(4, 2)$ |
| $4 = 2 \cdot 2$ | $\gcd(4, 2) = 2$ |

$\therefore \gcd(240, 46) = 2$

# 수학적 배경

## ➢ 합동식

- 두 정수 a, b에 대해 a-b가 m의 배수일 때, a와 b는 법(modular) m에 관하여 합동이라 하고, 다음과 같이 표시한다.

$$a \equiv b \mod m$$

- 즉, a-b = mk이다. (k는 임의의 정수)

- 합동식은 다음 세가지 성질을 만족한다.
  ① $a \equiv a \mod m$(반사성)
  ② $a \equiv b \mod m$ 이면, $b \equiv a \mod m$(대칭성)
  ③ $a \equiv b \mod m$, $b \equiv c \mod m$이면, $a \equiv c \mod m$(추이성)

# 수학적 배경

- 유한체($Z_n$)상에서의 곱셈에 대한 역원

$$a \cdot b \equiv 1 \bmod n$$

  - 위의 식을 만족하는 a와 b는 mod n상에서 곱셈에 대한 역원 관계에 있음
  - 단, (a, n)=1인 경우에만 곱셈에 대한 역원이 존재

- (a, n)=1인 정수 a, n에 대해,

$$a \cdot b + n \cdot x = 1$$

을 만족하는 b를 구하면, 이때 b가 mod n상에서 a의 곱셈에 대한 역원임

→ 확장 유클리드 알고리즘 이용

# 수학적 배경

➢ (예) mod 65상에서 27에 대한 곱셈의 역원 구하기

$27 \cdot x + 65 \cdot y = 1$을 만족하는 x 구하기

$65 = 27 \times 2 + 11$ (65를 27로 나누면 유일한 몫과 나머지 존재)
$27 = 11 \times 2 + 5$ (나누는 수(27)을 다시 나머지(11)로 나누기)
$11 = 5 \times 2 + 1$ (나누는 수(11)을 다시 나머지(5)로 나누기, 나머지가 1일 때 까지 반복)

$1 = 11 - 5 \times 2$ (나머지를 역순으로 정리)
$= 11 - (27 - 11 \times 2) \times 2 = 5 \times 11 - 2 \times 27$
$= 5 \times (65 - 27 \times 2) - 2 \times 27 = 5 \times 65 - 10 \times 27 - 2 \times 27$
$= 5 \times 65 - 12 \times 27$

$$65 \times 5 + 27 \times (-12) = 1 \quad \rightarrow \quad 27 \times (-12) + 65 \times 5 = 1$$

$$\therefore \; 27 \cdot (-12) \equiv 1 \bmod 65$$

$$27^{-1} \equiv -12 \equiv 53 \bmod 65$$

# 예제

➢ 확장 유클리드 알고리즘을 이용하여 mod 73상에서 25의 역원을 구하시오.

$$73 = 25 \cdot 2 + 23$$
$$25 = 23 \cdot 1 + 2$$
$$23 = 2 \cdot 11 + 1$$

$$\begin{aligned} 1 &= 23 - 2 \cdot 11 \\ &= 23 - (25 - 23 \cdot 1) \cdot 11 = 12 \cdot 23 - 11 \cdot 25 \\ &= 12(73 - 25 \cdot 2) - 11 \cdot 25 = 12 \cdot 73 - 35 \cdot 25 \\ &= 12 \cdot 73 + (-35) \cdot 25 \end{aligned}$$

$$\therefore\ 12 \cdot 73 + (-35) \cdot 25 = 1$$
$$25^{-1} \equiv -35 \equiv 38 \bmod 73$$

# 수학적 배경

## ➢ 완전 잉여계

- $Z_m = \{0, 1, 2, \cdots, m-1\}$

## ➢ 기약 잉여계

- $Z_m^* = \{a \in Z_m \mid \gcd(a, m) = 1\}$
- $Z_m$의 원소들 중 m과 서로 소인 것들의 집합

## ➢ 예제

- 완전 잉여계 $Z_9 = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$
- 기약 잉여계 $Z_9^* = \{1, 2, 4, 5, 7, 8\}$

# 수학적 배경

## ➢ Euler 함수

- 기약 잉여계 $Z_m^*$ 의 원소의 수를 오일러(Euler) 함수라 함
- $\varphi(m) = |Z_m^*|$

## ➢ Euler 함수 φ (m) 의 계산

- p가 소수이면, $\varphi(p) = p-1$
- $\varphi(m) = \varphi(p^s \times q^t \times r^u)$

$$= (p^s - p^{s-1})(q^t - q^{t-1})(r^u - r^{u-1})$$

(단 p, q, r : 소수)

# 수학적 배경

➢ 예제

$\varphi(7) = (7^1 - 7^0) = 6$

7은 소수이므로, $\varphi(7) = 7 - 1 = 6$

$\varphi(15) = \varphi(3^1 \times 5^1) = (3^1 - 3^{1-1})(5^1 - 5^{1-1})$

$= (3 - 1)(5 - 1) = 8$

$\varphi(9) = \varphi(3^2) = (3^2 - 3^1) = 6$

# 수학적 배경

❖ Euler의 정리

$$a^{\varphi(m)} \equiv 1 \bmod m \quad (\text{단, } (a, m) = 1)$$

❖ Fermat의 정리

$$a^{p-1} \equiv 1 \quad \bmod p \ (p : \text{소수})$$

➢ 예) m = 36일 때,

$\varphi(36) = \varphi(3^2 \times 2^2) = (3^2 - 3^1)(2^2 - 2^1)$

$= (9-3)(4-2) = 6 \times 2 = 12$

a = 7이라면, (7, 36)=1이므로,

$$7^{12} \equiv 1 \bmod 36$$

# RSA 암호 시스템

➢ RSA 암호화와 복호화는 $Z_n$상에서 모듈러 연산으로 수행

RSA 암호화 : 공개키 (n, e)와 평문 x에 대해 암호화 함수는 다음과 같다.

$$y \equiv x^e \bmod n \qquad x,y \in Z_n$$

RSA 복호화 : 개인키 d와 암호문 y에 대해 복호화 함수는 다음과 같다.

$$x \equiv y^d \bmod n \qquad x,y \in Z_n$$

- x, y, n, d는 1024비트 이상의 큰 수
- e : 암호화 지수(encryption exponent)
- d : 복호화 지수(decryption exponent)

# RSA 암호화와 복호화

## ➢ RSA 암호 시스템의 요구사항

1. 공격자도 공개키를 알 수 있기 때문에, **공개키 값 e와 n을 이용하여 개인키 d를 알아내는 것은 계산적으로 불가능**해야 한다.
2. x가 모듈러 n의 크기에 따라 결정되므로, 하나의 RSA 암호화로 l 비트 이상을 암호화 할 수 없다. (l은 n의 비트 길이)
3. $x^e$ mod n(암호화)과 $y^d$ md n(복호화)을 계산하는 것은 상대적으로 쉬워야 한다.
4. n에 대해 많은 개인키/공개키 쌍이 존재해야 한다.

# RSA 암호 방식의 키 생성

**[RSA 키 생성]**

**출력 : 공개키 $K_{pub}$ = (n, e) 및 개인키 $K_{pr}$=d**

1. 두 개의 큰 소수 p, q 선택
2. n = p • q 계산
3. $\phi$(n) = (p – 1) (q – 1) 계산
4. gcd (e, $\phi$(n)) = 1인 공개키 e 선택(e∈{1, 2, …, $\phi$(n)-1}
5. 다음을 만족하는 개인키 d 계산

$$d \cdot e \equiv 1 \bmod \phi(n)$$

# RSA 암호 방식 예제

[예제]

- Alice가 Bob에게 암호화된 메시지를 보내고자 한다. 먼저 Bob은 RSA 파라미터를 계산한다. 그리고 난 후, Bob은 자신의 공개키를 Alice에게 전송한다. Alice는 메시지(x=4)를 암호화하여 암호문 y를 Bob에게 전송한다. Bob은 자신의 개인키를 이용하여 y를 복호한다.

- Bob이 파라미터 p=3, q=11을 선택한 경우,

# RSA 암호 방식 예제

**Alice**

메시지 x=4

**Bob**

p=3, q=11 선택
n=p•q=33
$\phi(n)$ = (3-1)(11-1)=20
e=3 선택
$d \equiv e^{-1} \equiv 7 \bmod 20$

공개키 = (33, 3) ←

메시지 x=4에 대한 암호화
$y = 4^3 \bmod 33$
$\equiv 31 \bmod 33$

암호문 = 31 →

복호화
$x = 31^7 \bmod 33$
$\equiv 4 \bmod 33$

# RSA 암호 방식의 증명

$$M^{ed} \equiv M \bmod n$$

- $d \cdot e \equiv 1 \bmod \phi(n)$
- $d \cdot e \equiv 1 + t \cdot \phi(n)$ (t는 임의의 정수)

- $M \equiv M^{ed} \bmod n$

  $\equiv M^{1 + t \cdot \phi(n)} \bmod n$

  $\equiv M^1 \cdot (M^{\phi(n)})^t \bmod n$ (Euler 정리에 의해)

  $\equiv M \cdot (1)^t \bmod n$

  $\equiv M \bmod n$ (단, $(M, n) = 1$ )

$M^{\phi(n)} \equiv 1 \bmod n$
(M과 n인 서로 소인 경우)

# RSA 암호 시스템

## [키 생성]

① Alice와 Bob은 각각 자신의 개인키/공개키를 생성한다.

② Alice와 Bob은 자신의 공개키와 모듈러 값을 공개한다.

## [암호화 및 복호화 과정]

- Alice가 Bob에게 메시지 M을 암호화 해서 보내려는 경우,

① Alice는 Bob의 공개키 $e_B$와 모듈러 값 $n_B$을 획득한다.

② Alice는 다음과 같이 암호문을 생성하여 Bob에게 전송한다.

$$C \equiv M^{e_B} \bmod n_B$$

③ Bob은 자신의 개인키 $d_B$ 를 이용하여 암호문을 복호화한다.

$$M \equiv C^{d_B} \bmod n_B$$

# RSA 암호 시스템

## ➢ 암호화 및 복호화 과정

**공개 정보**
**$(e_A, n_A)$ , $(e_B, n_B)$**

Alice

Bob

$\xrightarrow{\quad C \quad}$

$n_A = p_A \times q_A$
$e_A$, $d_A$ 계산

$n_B = p_B \times q_B$
$e_B$, $d_B$ 계산

**암호화**
$C \equiv M^{e_B} \bmod n_B$

**복호화**
$M \equiv C^{d_B} \bmod n_B$

# RSA 암호 방식의 예

## ➢ 키 생성

- $p = 3,\ q = 11$
- $n = 33,\ \phi(33) = (3-1)(11-1) = 20$
- 공개키 e 선택 : gcd $(e, \phi(33)) = 1$인 e =3 선택
- 개인키 d 계산 : $e \times d \equiv 1 \mod \phi(33)$

  $d = 7$

## ➢ 암호화/복호화

- M = 5인 경우,
- 암호화 : $C \equiv M^e \mod n \equiv 5^3 \mod 33 \equiv 26$
- 복호화 : $M \equiv C^d \mod n \equiv 26^7 \mod 33 \equiv 5$

# RSA 암호 방식의 예

- 소수 p=17, q=19를 사용하는 RSA 암호 시스템에서 공개키를 157로 사용할 때, 대칭되는 개인키를 구하시오.

$n = p \cdot q = 17 \cdot 19 = 323$
$\phi(n) = (p-1)(q-1) = 16 \cdot 18 = 288$

공개키 $e = 157$일 때, $e \cdot d \equiv 1 \bmod \phi(n)$을 만족하는 d 구하기

$157 \cdot a + 288 \cdot b = 1$ 을 만족하는 a가 개인키 d가 됨

$288 = 157 \cdot 1 + 131$
$157 = 131 \cdot 1 + 26$
$131 = 26 \cdot 5 + 1$

$$\begin{aligned} 1 &= 131 - 26 \cdot 5 \\ &= 131 - (157 - 131 \cdot 1)5 = 6 \cdot 131 - 5 \cdot 157 \\ &= 6(288 - 157 \cdot 1) - 5 \cdot 157 \\ &= -11 \cdot 157 + 6 \cdot 288 \end{aligned}$$

$-11 \bmod 288 \equiv 277 \bmod 288$ 이므로, 개인키 $d = 277$

# RSA 암호 방식의 예

- [예제 5.1] 소수 p=47, q=59로 RSA 공개키 암호 방식을 구성해보자.(교재 p.175)
  - $n = pq = 2773$
  - $\phi(n) = (p-1)(q-1) = 46 \cdot 58 = 2668$
  - gcd(e, 2668)=1인 e 선택 : 157 선택
  - $e \cdot d \equiv 1 \bmod 2668$을 만족하는 d 계산

$$2668 = 157 \times 16 + 156$$
$$157 = 156 \times 1 + 1$$

$$\begin{aligned} 1 &= 157 - 156 \times 1 \\ &= 157 - (2668 - 16 \times 157) \\ &= 17 \times 157 + (-1) \times 2668 \end{aligned}$$

$$\therefore 157 \times 17 \equiv 1 \bmod 2668$$

# 4.3 RSA – 제곱의 반복

## ➢ $5^{20}$을 계산하는 예

- 간단하게 5를 20번 곱해서 그 결과를 모듈로 35로 줄이면 다음과 같음

$$5^{20} = 95{,}367{,}431{,}640{,}625 = 25 \pmod{35} \qquad (4.4)$$

- RSA 암호화 $C = M^e \pmod{N}$ 또는 복호화 $M = C^d \pmod{N}$을 계산한다고 가정
- 안전하게 RSA를 실행시키면, 계수 $N$은 적어도 1024비트

- 지수 20은 이진수로 10100
- 지수 10100은 고차 비트에서 시작해 한 번에 한 비트씩 다음과 같이 만들어질 수 있음

$$(0, 1, 10, 101, 1010, 10100) = (0, 1, 2, 5, 10, 20)$$

- 지수 20은 다음과 같은 단계로 구성

$$1 = 0 \cdot 2 + ①$$
$$2 = 1 \cdot 2 + ⓪$$
$$5 = 2 \cdot 2 + ①$$
$$10 = 5 \cdot 2 + ⓪$$
$$20 = 10 \cdot 2 + ⓪$$

# 4.3 RSA – 제곱의 반복

- 제곱의 반복을 이용해 $5^{20}$을 계산하기 위해 이 알고리즘을 지수에 적용

$$5^1 = 1^2 \cdot 5^{①} = 1^2 \cdot 5 = 5 = 5 \pmod{35}$$

$$5^2 = 5^2 \cdot 5^{⓪} = 5^2 \cdot 1 = 25 = 25 \pmod{35}$$

$$5^5 = 25^2 \cdot 5^{①} = 25^2 \cdot 5 = 3125 = 10 \pmod{35}$$

$$5^{10} = 10^2 \cdot 5^{⓪} = 10^2 \cdot 1 = 100 = 30 \pmod{35}$$

$$5^{20} = 30^2 \cdot 5^{⓪} = 30^2 \cdot 1 = 900 = 25 \pmod{35}$$

# RSA 지수 승의 가속화

- 공개키 알고리즘은 매우 큰 수의 산술 연산에 기반
- 연산을 효율적으로 구현하지 못하면 실제로 사용하기에 어려움이 있음
- RSA 암호화/복호화는 모두 모듈러 지수승 수행
  - $y \equiv x^e \mod n$ (암호화)
  - $x \equiv y^d \mod n$ (복호화)

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \ldots$$

(SQ : 제곱, MUL : 곱셈)

- 1024비트의 e, d를 사용하는 경우, $2^{1024}$번의 곱셈을 하는 것은 적절치 않음

  -> 지수 승을 빠르게 하는 방법 필요

# RSA 지수 승의 가속화

## [예제] $x^8$ 계산

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \xrightarrow{MUL} x^6 \xrightarrow{MUL} x^7 \xrightarrow{MUL} x^8$$

-> 1번의 제곱과 7번의 곱셈연산 필요

- 빠른 방법

$$x \xrightarrow{SQ} x^2 \xrightarrow{SQ} x^4 \xrightarrow{SQ} x^8$$

-> 3번의 제곱연산 필요

- 빠른 방법이 좋지만, 멱승이 2로 제한됨
- 공개키/개인키가 $2^i$ 형태일 때만 가능

# RSA 지수 승의 가속화

## [예제] $x^{26}$ 계산

- 일반적인 방법 : 25번의 곱셈
- 보다 빠른 방법 : 2번의 곱셈과 4번의 제곱(6번 연산)

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{SQ} x^6 \xrightarrow{SQ} x^{12} \xrightarrow{MUL} x^{13} \xrightarrow{SQ} x^{26}$$

$$x^{26} = x^{11010} = x^{h_4 h_3 h_2 h_1 h_0}$$

- 지수를 2진수(비트)로 표현한 뒤, 왼쪽(최상위비트)부터 오른쪽(최하위비트) 순서로 연산 수행

# RSA 지수 승의 가속화

## ➢ 모듈러 지수승을 위한 제곱 후 곱셈 알고리즘

- 입력 : x, H, n($H=h_t h_{t-1} \ldots h_1 h_0$, $h_t=1$)
- 출력 : $x^H$ mod n
- 초기화 : r = x

[알고리즘 ]

1. FOR i=t-1 DOWNTO 0
1.1 $r = r^2 \bmod n$
IF $h_i$ =1
1.2 $r = r \cdot x \bmod n$
2. RETURN(r)

# RSA 지수 승의 가속화

## ➢ 제곱 후 곱셈(Square-and-multiply) 알고리즘

$$x^{26} = x^{11010} = x^{h_4 h_3 h_2 h_1 h_0}$$

1 2 3 4

| 단계 | 연산 | |
|---|---|---|
| 0 | $x=x^1$ | 초기설정, $h_4 = 1$ |
| 1a | $(x^1)^2 = x^2$ $(x^{10})$ | SQ 수행 |
| 1b | $x^2 \cdot x = x^3$ $(x^{11})$ | $h_3 = 1$이므로, MUL 수행 |
| 2a | $(x^3)^2 = x^6$ $(x^{110})$ | SQ 수행 |
| 2b | | $h_2 = 0$이므로, MUL 수행하지 않음 |
| 3a | $(x^6)^2 = x^{12}$ $(x^{1110})$ | SQ 수행 |
| 3b | $x^{12} \cdot x = x^{13}$ $(x^{1101})$ | $h_1 = 1$이므로, MUL 수행 |
| 4a | $(x^{13})^2 = x^{26}$ $(x^{11010})$ | SQ 수행 |
| 4b | | $h_0 = 0$이므로, MUL 수행하지 않음 |

# RSA 지수 승의 가속화

## [예제] 제곱 후 곱셈 알고리즘을 사용하는 경우, $x^{35}$ 의 연산 회수를 구하시오.

$$35 = 100011_2, \quad x^{35} = x^{100011} = x^{h_5 h_4 h_3 h_2 h_1 h_0}$$

5번의 제곱과
2번의 곱셈 수행

| 단계 | 연산 | |
|---|---|---|
| 0 | $x=x^1$ | 초기설정, $h_5 = 1$ |
| 1a | $(x^1)^2 = x^2 \quad (x^{10})$ | SQ 수행 |
| 1b | | $h_4 = 0$이므로, MUL 수행하지 않음 |
| 2a | $(x^2)^2 = x^4 \quad (x^{100})$ | SQ 수행 |
| 2b | | $h_3 = 0$이므로, MUL 수행하지 않음 |
| 3a | $(x^4)^2 = x^8 \quad (x^{1000})$ | SQ 수행 |
| 3b | | $h_2 = 0$이므로, MUL 수행하지 않음 |
| 4a | $(x^8)^2 = x^{16} \quad (x^{10000})$ | SQ 수행 |
| 4b | $x^{16} \cdot x = x^{17} \quad (x^{10001})$ | $h_1 = 1$이므로, MUL 수행 |
| 4a | $(x^{17})^2 = x^{34} \quad (x^{100010})$ | SQ 수행 |
| 4b | $x^{34} \cdot x = x^{35} \quad (x^{100011})$ | $h_0 = 1$이므로, MUL 수행 |

# RSA 지수 승의 가속화

## ➢ 짧은 공개 지수를 이용한 빠른 암호화

- 실제 응용에서 RSA의 공개키 e를 매우 작은 값으로 선택
- 공개키의 비트 표현에서 1의 개수가 적을 수록 연산 회수가 적음
- e가 짧아도 개인키 d는 충분히 길어야 함

[짧은 공개지수를 이용하는 RSA 지수 승의 복잡도]

| 공개키 e | e의 이진 표현 | #MUL + #SQ |
|---|---|---|
| 3 | 11 | 3 |
| 17 | 1001 | 5 |
| $2^{16}+1$ | 10000000000000001 | 17 |

# RSA 암호 방식의 안전성

## ➢ 소수 p, q의 선택 조건

- $p \approx q$ : p와 q의 크기는 거의 같아야 함
- (p-1)과 (q-1)은 큰 소수를 인수로 가져야 함
  - $p = 2p' + 1$ $p'$ : 소수
  - $q = 2q' + 1$ $q'$ : 소수
- (p-1)과 (q-1)의 최대 공약수는 작아야 함

# RSA 암호 방식의 안전성

- 공개키 암호 방식은 공개키로부터 개인키를 구하는 것이 불가능해야 함

- RSA 암호 시스템에서 공개키 (e, n)을 이용하여 개인키 d를 구하는 것이 불가능해야 함

  - (e, n)으로 부터 d를 구하기 위해서는 $\phi(n)$ 값을 알아야 함
  - $\phi(n)=(p-1)(q-1)$이므로, p와 q를 구해야 함
  - 즉, n을 소인수 분해 할 수 있다면 RSA 암호 시스템은 쉽게 해독됨

- RSA 암호 방식의 안전성은 소수 p, q에 의존

# RSA 암호 방식의 안전성

- RSA 인수분해 기록

| 자릿수 | 비트 길이 | 날짜 |
|---|---|---|
| 100 | 330 | 1991년 4월 |
| 110 | 364 | 1992년 4월 |
| 120 | 397 | 1993년 6월 |
| 129 | 426 | 1994년 4월 |
| 140 | 463 | 1999년 2월 |
| 155 | 512 | 1999년 8월 |
| 200 | 664 | 2005년 5월 |

- 장기간의 안전성을 위해서라면 2048비트 이상의 RSA 파라미터를 선택하는 것이 바람직

# RSA 암호 방식의 안전성

- $\phi(n)$을 알게되는 경우, p와 q를 구할 수 있음

$$n = p \cdot q$$
$$\phi(n) = (p-1)(q-1)$$

q = n/p을 두 번째 식에 대입하면, 다음과 같은 p에 대한 2차 방정식을 얻을 수 있음

$$p^2 - (n - \phi(n) + 1)p + n = 0$$

이 방정식의 두 개의 근이 p, q가 된다

- $\phi(n)$이 노출되어서는 안되며, 키 생성 후 삭제 가능

# RSA 암호 방식의 안전성

- [예제] p=11, q=3으로 구성되는 작은 n값의 RSA 암호 방식에 대해 반복 암호화 공격을 수행해보자.
  - $n = pq = 33$
  - $\phi(n) = (11-1)(3-1) = 20$
  - 공개키 $e = 3$ 선택
  - 개인키 $d = 7$ ($e \cdot d \equiv 1 \bmod 20$)
  - 평문 m=3에 대한 암호화 : $C \equiv m^e \bmod 33 \equiv 3^3 \bmod 33 \equiv 27 \bmod 33$
  - 암호문 C에 대한 반복 암호화

    $27^3 \equiv 15 \bmod 33$ $\quad 15^3 \equiv 9 \bmod 33$ $\quad 9^3 \equiv 3 \bmod 33$ $\quad 3^3 \equiv 27 \bmod 33$

안전한 RSA 암호 방식을 구성하기 위해서는 p, q의 선택이 중요함

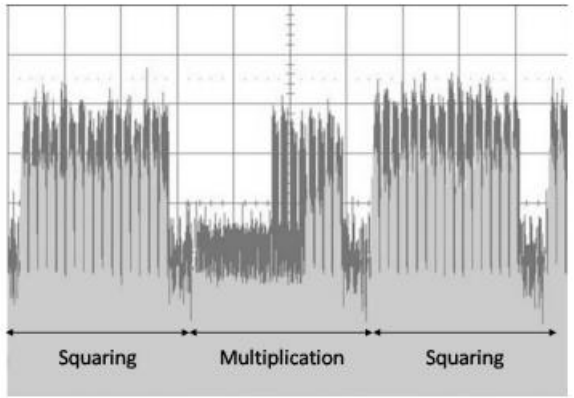
# RSA 암호 방식의 안전성

## ➢ 부채널 공격

- 전력 소모나 시간 반응과 같은 물리적 채널을 통해 유출되는 정보를 통해 개인키를 유추하는 공격
- RSA 지수승을 제곱 후 곱셈 알고리즘으로 구현한 경우, 지수 비트의 값이 1인 경우와 0인 경우에 소모되는 전력량에 차이가 있음

➢ 연산 : S SM SM S SM S S SM SM SM S SM

➢ 개인키 : 0 1 1 0 1 0 0 1 1 1 0 1




➢ 지수 비트가 0인 경우 제곱 후 더미 값을 갖는 곱셈을 수행하여 방지

# RSA의 실제 사용

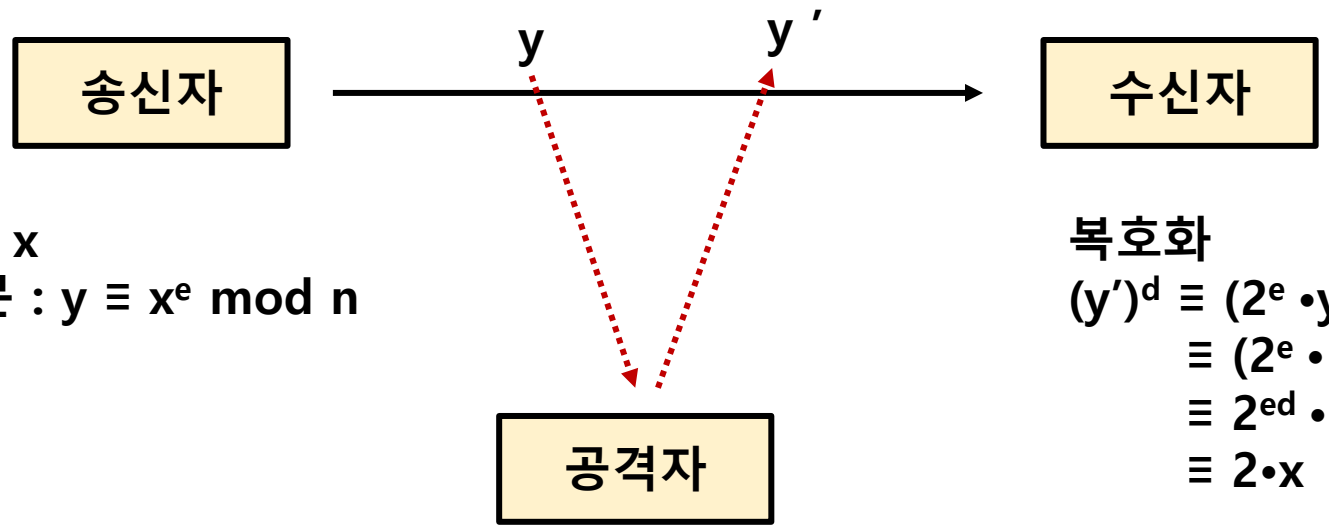
- RSA 암호 방식은 결정적(deterministic)임
  - 특정 키에 대해 특정 평문은 항상 동일한 암호문 생성

- RSA 암호 방식은 변환 가능(malleable)함
  - 공격자는 암호문을 복호화하지는 못하지만, 복호화 시 다른 평문이 나오도록 조작가능

  - 암호문 : $y \equiv x^e \bmod n$
  - 평문 x가 금액과 같은 숫자일 때, 공격자는 x를 알지는 못하지만 2배 증가시킬 수 있음
  - 전송되는 암호문을 가로챈 후, 암호문에 $2^e$을 곱해서 전송하면, 수신자가 복호화하면 2배 증가한 값을 얻게 됨

# RSA의 실제 사용

- 수신자의 공개키 : (n, e)
- 수신자의 비밀키 : d




평문 : x
암호문 : $y \equiv x^e \bmod n$

복호화
$(y')^d \equiv (2^e \cdot y)^d \bmod n$
$\equiv (2^e \cdot x^e)^d \bmod n$
$\equiv 2^{ed} \cdot x^{ed} \bmod n$
$\equiv 2 \cdot x \bmod n$

암호문 변조
$y' \equiv 2^e \cdot y \bmod n$

송신자가 보낸 평문 x의
2배가 되는 값 획득

- 공격자는 암호문을 복호화하지는 못하지만, 송신자가 보낸 값이 2배 증가하도록 조작가능(malleable)
- 해결책 : 암호화 하기 전 평문에 랜덤한 구조를 추가하는 OAEP(Optimal Asymmetric Encryption Padding) 방식 사용

# 정리

- RSA는 가장 널리 사용되는 공개키 암호 시스템으로, 키 전송(키 암호화)과 전자서명에 주로 사용
- 공개키 e는 짧은 정수일 수 있으며, 개인키 d는 모듈러 n의 길이만큼 길어야 함
- RSA의 안전성은 정수의 인수분해 문제에 기반하므로, 현재 컴퓨팅 시스템으로 인수분해 하기에 어려운 충분히 긴 모듈러 n을 사용해야 함
- 장기간의 안전성을 위해서는 2048비트 이상의 모듈러를 사용할 것을 권장
- RSA에 대한 주요 공격 방법은 인수분해, 부채널 공격 등이 있음